

+++ date = '2026-04-02T00:00:00-08:00' draft = false title = 'Practica 3: Programacion funcional. Haskell' +++

Práctica 3: Instalación de Entorno y Aplicación TODO en Haskell

Subsecciones

- Reporte
 - Introducción
 - Instalación del entorno
 - GHCup
 - GHC
 - Stack
 - Cabal
 - HLS
 - Verificación del entorno
 - Introducción a Haskell
 - Sintaxis básica
 - Paradigma funcional
 - Funciones y listas
 - Aplicación TODO en Haskell
 - Estructura del proyecto
 - Funcionamiento
 - Manejo de tareas
 - Operaciones IO
 - Ventajas y desventajas de Haskell
 - Preguntas guía
 - Conclusiones
 - Referencias
- Sesiones:
 - Sesión 1
 - Sesión 2

1. Introducción

En esta práctica se realizó la instalación del entorno de desarrollo para el lenguaje de programación Haskell utilizando GHCup, además de explorar las herramientas principales del ecosistema de desarrollo.

Posteriormente, se estudió el funcionamiento de una aplicación TODO desarrollada en Haskell utilizando Stack, con el objetivo de comprender la estructura de proyectos, la sintaxis básica del lenguaje y el paradigma funcional.

Haskell es un lenguaje de programación puramente funcional que se caracteriza por su tipado fuerte, evaluación perezosa y enfoque matemático. Aunque su sintaxis resulta diferente a lenguajes imperativos tradicionales como C o Python, permite desarrollar software modular, seguro y altamente mantenible.

2. Instalación del entorno

2.1 GHCup

La instalación del entorno de Haskell se realiza mediante GHCup, una herramienta que automatiza la descarga e instalación de todos los componentes necesarios.

Sitios oficiales:

- <https://www.haskell.org/downloads/>
- <https://www.haskell.org/ghcup/>
- <https://www.haskell.org/get-started/>

Procedimiento

1. Ingresar al sitio oficial de descargas de Haskell.
2. Seleccionar la opción de instalación mediante GHCup.
3. Abrir PowerShell en Windows sin permisos de administrador.
4. Copiar y ejecutar el comando proporcionado por GHCup.

Ejemplo del comando:

```
Set-ExecutionPolicy Bypass -Scope Process -Force; `
[System.Net.ServicePointManager]::SecurityProtocol = `
[System.Net.ServicePointManager]::SecurityProtocol -bor 3072; `
& ([scriptblock]::Create((Invoke-WebRequest
https://www.haskell.org/ghcup/sh/bootstrap-haskell.ps1).Content))
```

Durante la instalación se descargan automáticamente las herramientas principales del entorno.

2.2 GHC

GHC (Glasgow Haskell Compiler) es el compilador principal de Haskell.

Su función es transformar el código fuente `.hs` en programas ejecutables.

Ejemplo para verificar instalación:

```
ghc --version
```

2.3 Stack

Stack es un administrador de proyectos y dependencias para Haskell.

Permite:

- Crear proyectos
- Descargar librerías
- Compilar programas
- Ejecutar aplicaciones

Verificación:

```
stack --version
```

2.4 Cabal

Cabal es una herramienta de construcción y empaquetado para proyectos Haskell.

Funciones principales:

- Gestión de dependencias
- Compilación
- Distribución de paquetes

Verificación:

```
cabal --version
```

2.5 HLS

Haskell Language Server (HLS) proporciona soporte para editores de código:

- Autocompletado
- Diagnóstico de errores
- Refactorización
- Navegación de código

Es utilizado por editores como Visual Studio Code.

3. Verificación de entorno

Después de instalar todas las herramientas, se realizó una prueba básica creando un archivo llamado:

```
Main.hs
```

Contenido:

```
main = putStrLn "Hola Mundo desde Haskell"
```

Compilación:

```
ghc Main.hs
```

Ejecución:

```
./Main
```

Resultado esperado:

```
Hola Mundo desde Haskell
```

4. Introduccion a Haskell

4.1 Sintaxis basica

Haskell utiliza una sintaxis minimalista y orientada a funciones.

Ejemplo:

```
suma a b = a + b
```

Uso:

```
suma 5 3
```

Resultado:

```
8
```

4.2 Paradigma funcional

Haskell pertenece al paradigma funcional.

Características principales:

Variables inmutables Uso intensivo de funciones Ausencia de efectos secundarios Evaluación perezosa
Funciones de orden superior

Ejemplo recursivo:

```
factorial 0 = 1
factorial n = n * factorial (n - 1)
```

4.3 Funciones y listas

Las listas son estructuras fundamentales en Haskell.

Ejemplo:

```
numeros = [1,2,3,4,5]
```

Concatenación:

```
[1,2] ++ [3,4]
```

Resultado:

```
[1,2,3,4]
```

5. Aplicacion TODO en Haskell

5.1 Estructura del proyecto

La aplicación TODO se desarrolló utilizando Stack.

Creación del proyecto:

```
stack new todo
```

Estructura básica:

```
todo/
|
├─ app/
|   └─ Main.hs
```

```
|  
├─ package.yaml  
├─ stack.yaml  
└─ README.md
```

5.2 Funcionamiento

La aplicación TODO permite:

Agregar tareas Mostrar tareas Eliminar tareas Guardar información de tareas

El objetivo principal es demostrar el manejo de listas, funciones e interacción con el usuario en Haskell.

5.3 Manejo de tareas

Representación de tareas:

```
type Task = String
```

Agregar tareas:

```
addTask tasks newTask = tasks ++ [newTask]
```

Explicación:

- tasks representa la lista actual
- newTask es la tarea agregada
- ++ concatena listas

5.4 Mostrar tareas

```
showTasks tasks = mapM_ putStrLn tasks
```

Explicación:

- mapM_ ejecuta una acción sobre cada elemento
- putStrLn imprime cada tarea

5.5 Eliminar tareas

```
removeTask tasks index =  
    take index tasks ++ drop (index + 1) tasks
```

Explicación:

- **take** conserva elementos anteriores
- **drop** elimina elementos posteriores

Operaciones IO

La interacción con el usuario en Haskell se realiza mediante IO.

Ejemplo:

```
main = do
  putStrLn "Ingrese una tarea:"
  tarea <- getLine
  putStrLn ("Tarea agregada: " ++ tarea)
```

Explicación:

- do agrupa acciones IO
- getLine obtiene texto del usuario
- <- extrae valores de operaciones IO

6. Ventajas y desventajas de Haskell

Ventajas

- Tipado fuerte y seguro
- Código modular
- Facilita mantenimiento
- Evaluación perezosa
- Paradigma funcional moderno

Desventajas

- Curva de aprendizaje elevada
- Sintaxis poco familiar
- Menor cantidad de recursos comparado con otros lenguajes
- Menor demanda laboral en comparación con Java o Python

8. Conclusiones

La práctica permitió instalar correctamente el entorno de desarrollo de Haskell utilizando GHCup y comprender las herramientas principales del ecosistema del lenguaje.

Además, se logró analizar una aplicación TODO desarrollada con Stack, observando cómo Haskell maneja listas, funciones y operaciones IO mediante el paradigma funcional.

Aunque Haskell presenta una sintaxis diferente a los lenguajes imperativos tradicionales, proporciona ventajas importantes relacionadas con seguridad, modularidad y claridad matemática.

Finalmente, esta práctica sirvió como introducción al paradigma funcional y permitió conocer la estructura básica de proyectos desarrollados en Haskell.

9. Referencias

Downloads. (s. f.). <https://www.haskell.org/downloads/>

Team, G. (s. f.). GHCUP. <https://www.haskell.org/ghcup/>

Get started. (s. f.). <https://www.haskell.org/get-started/>

Contributors, S. (s. f.). Stack. <https://docs.haskellstack.org/en/stable/>

Steadylearner. (s. f.). Haskell/examples/blog/todo at main · steadylearner/Haskell. GitHub. <https://github.com/steadylearner/Haskell/tree/main/examples/blog/todo>

Haskell Tutorial for C Programmers - HaskellWiki. (s. f.). https://wiki.haskell.org/index.php?title=Haskell_Tutorial_for_C_Programmers

Sesiones

Sesión 1

Instalación de herramientas

Verificación de GHC

```
ghc --version
```

Verificación de Stack

```
stack --version
```

Verificación de Cabal

```
cabal --version
```

Primer programa en Haskell

main.hs


```
main = do
  putStrLn "Hello, everybody!"
  putStrLn ("Please look at my favorite odd numbers: " ++ show (filter odd
[10..20]))
```

Compilacion

```
ghc Main.hs
```

Ejecucion

```
./Main
```

 instalacion instalacion2 firstlines firstprogram

Sesion 2

Creacion del proyecto TODO

Crear proyecto

```
stack new todo
```

Entrar al directorio

```
cd todo
```

Compilar proyecto

```
stack build
```

Ejecutar aplicacion

```
stack run
```

Ejemplo de manejo de tareas

Agregar tarea

```
addTask tasks newTask = tasks ++ [newTask]
```

Mostrar tareas

```
showTasks tasks = mapM_ putStrLn tasks
```

Eliminar tarea

```
removeTask tasks index =  
  take index tasks ++ drop (index + 1) tasks
```

Ejemplo de IO

```
main = do  
  putStrLn "Ingrese una tarea:"  
  tarea <- getLine  
  putStrLn ("Tarea agregada: " ++ tarea)
```

 aplicaciontodo1

 aplicaciontodo2